CDC

EsportHub

1. Contexte & périmètre

1.1 Contexte

EsportHub est une API dédiée aux fans d'esport FPS tactique. Elle centralise les informations essentielles sur les compétitions professionnelles : équipes, joueurs, matchs en direct, classements et actualités.

Le projet propose des fonctionnalités communautaires (likes, favoris, comptes utilisateurs). Elle a vocation à être consommée par n'importe quel client : application mobile, web, bot Discord, etc.

L'architecture est pensée dès le départ pour être multi-jeux dans le périmètre des FPS tactiques : bien que le lancement soit centré sur Valorant, le modèle de données est conçu pour accueillir d'autres titres du même genre (CS2, Rainbow Six Siege...) sans refonte majeure. Les mécaniques communes à ces jeux (rounds CT/T, maps, économie) sont prises en compte nativement dans le modèle.

1.2 Périmètre

Inclus	Exposition de données esport, gestion de compte utilisateur, likes/favoris, suivi live, classements, actualités, événements
Exclu	Streaming vidéo, paris, messagerie, gestion de tournois, contenu payant, jeux hors FPS tactique (MOBA, RTS, Battle Royale...)
Format	API REST, réponses JSON, authentification JWT
Jeux ciblés	Valorant (V1) — CS2, Rainbow Six Siege (V2+)

2. Cibles & utilisateurs

Consommateurs de l'API

Développeur front-end / mobile — Consomme les endpoints pour construire une interface utilisateur (app mobile, site web). C'est le consommateur

principal.

Bot / intégration tierce — Consomme les endpoints publics (matchs live, classements, résultats) pour alimenter un bot Discord, une extension navigateur, un widget, etc.

Rôles utilisateurs (côté authentification)

Visiteur (non authentifié) — Accès en lecture seule aux données publiques : équipes, joueurs, matchs, événements, news. Aucun accès aux endpoints communautaires.

Utilisateur authentifié — Accès complet : gestion de son compte, likes d'équipes, suivi de joueurs, consultation de ses favoris.

---

3. Fonctionnalités & endpoints dans les grandes lignes

3.1 Authentification

POST /auth/register

- Inscription email + mot de passe
- Validation du format email, force du mot de passe
- Retourne un access token + refresh token
- Erreurs : email déjà utilisé, mot de passe trop faible

POST /auth/login

- Login email + mot de passe
- Retourne un access token + refresh token

POST /auth/refresh

- Renouvellement du access token via refresh token

POST /auth/logout

- Révocation du refresh token

3.2 Compte utilisateur

GET /users/me — Retourne le profil de l'utilisateur connecté (username, avatar, email, date d'inscription)

PATCH /users/me — Modification du username et/ou de l'avatar

PATCH /users/me/password — Changement de mot de passe (ancien + nouveau requis)

PATCH /users/me/email — Changement d'email (mot de passe requis pour confirmation)

DELETE /users/me — Suppression du compte et de toutes les données associées

GET /users/me/likes — Liste des équipes likées par l'utilisateur

GET /users/me/follows — Liste des joueurs suivis par l'utilisateur

---

3.3 Global Esport

GET /games — Liste des jeux FPS tactiques supportés (slug, nom, logo)

GET /matches/live — Tous les matchs en cours, filtrables par jeu (`?game=valorant`). Score, équipes, map en cours, tournoi.

GET /news — Feed des dernières actualités, filtrable par jeu. Titre, description, auteur, date, lien.

3.4 Équipes

GET /teams — Liste des équipes. Paramètres : `?game=`, `?region=`, `?sort=rank|earnings`, `?page=`, `?limit=`

GET /teams/search — Recherche par nom. Paramètre : `?q=`

GET /teams/:id — Fiche équipe : nom, logo, région, rang, cashprize total, compteur de likes

POST /teams/:id/like — Liker une équipe *(authentifié)*

DELETE /teams/:id/like — Retirer son like *(authentifié)*

GET /teams/:id/matches/live — Matchs en cours de l'équipe : score temps réel, map

GET /teams/:id/matches — Historique des matchs de l'équipe : score, adversaire, tournoi, date

GET /teams/:id/players — Roster actuel : liste des joueurs avec rôle et lien vers leur fiche

GET /teams/:id/transactions — Historique des arrivées/départs : joueur, type (join/leave), date

GET /rankings — Classement des équipes par région. Paramètres : `?region=`, `?game=`. Retourne rang, record, earnings, indicateur d'évolution.

3.5 Matchs

GET /matches — Liste des matchs. Paramètres : `?status=live|upcoming|completed`, `?game=`, `?page=`

GET /matches/:id — Détail d'un match : équipes, score map par map, statut, tournoi, phase, date

GET /matches/:id/stats — Stats des joueurs pour ce match : ACS, K/D, KAST, ADR, KPR, HS%, Clutch%

3.6 Joueurs

GET /players — Classement des joueurs. Paramètres : `?region=`, `?sort=rating|acs|kd`, `?game=`

GET /players/search — Recherche par pseudo. Paramètre : `?q=`

GET /players/:id — Fiche joueur : pseudo, organisation, nationalité, compteur de followers

POST /players/:id/follow — Suivre un joueur *(authentifié)*

DELETE /players/:id/follow — Ne plus suivre un joueur *(authentifié)*

GET /players/:id/stats — Stats du joueur : Rating, ACS, K/D, KAST, ADR, KPR, HS%, Clutch%. Paramètres : `?region=`

GET /players/:id/teams — Historique des équipes avec période d'appartenance

GET /players/:id/events — Événements joués : nom, placement, date

GET /players/:id/matches — Matchs joués : adversaire, résultat, score, date

---

3.7 Événements

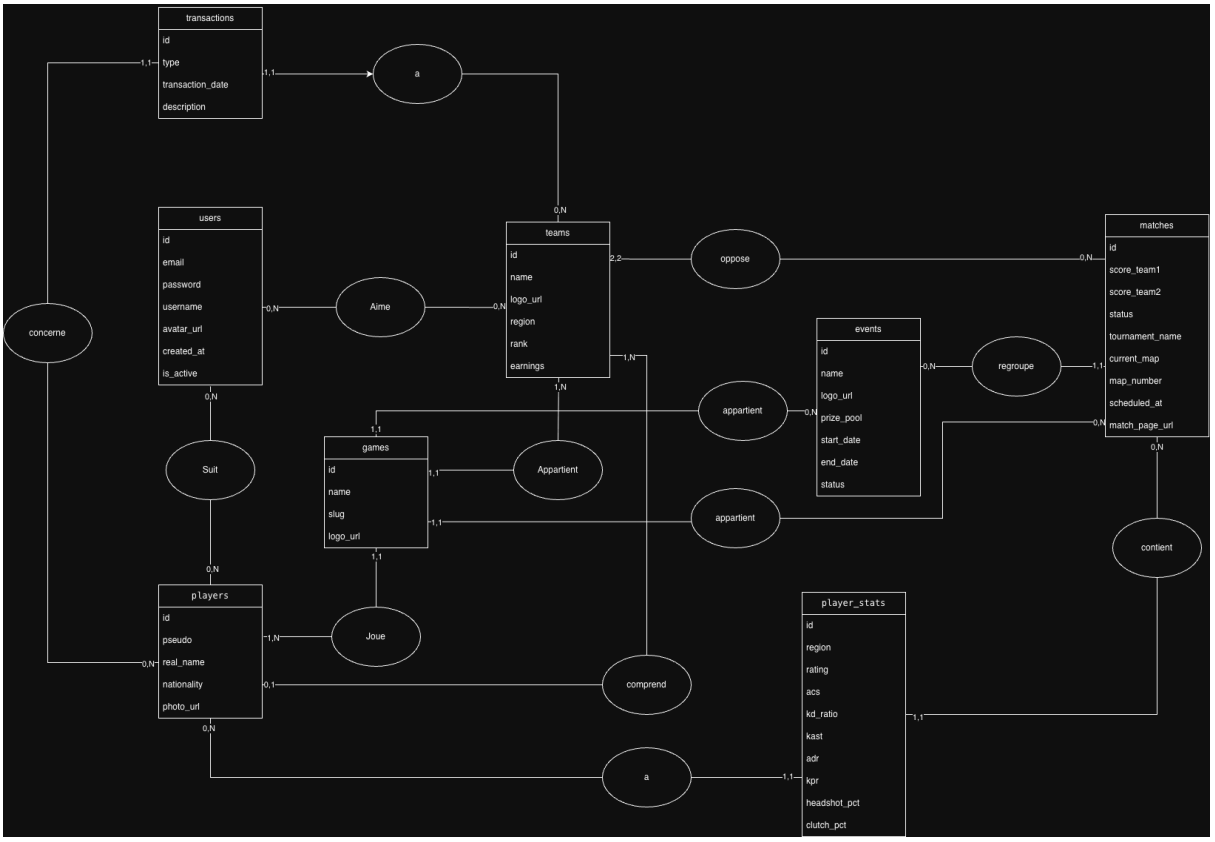
GET /events — Liste des événements. Paramètres : `?status=upcoming|ongoing|completed`, `?game=`, `?page=`

GET /events/:id — Fiche événement : nom, logo, prize pool, dates, région

GET /events/:id/matches — Matchs associés à l'événement

---

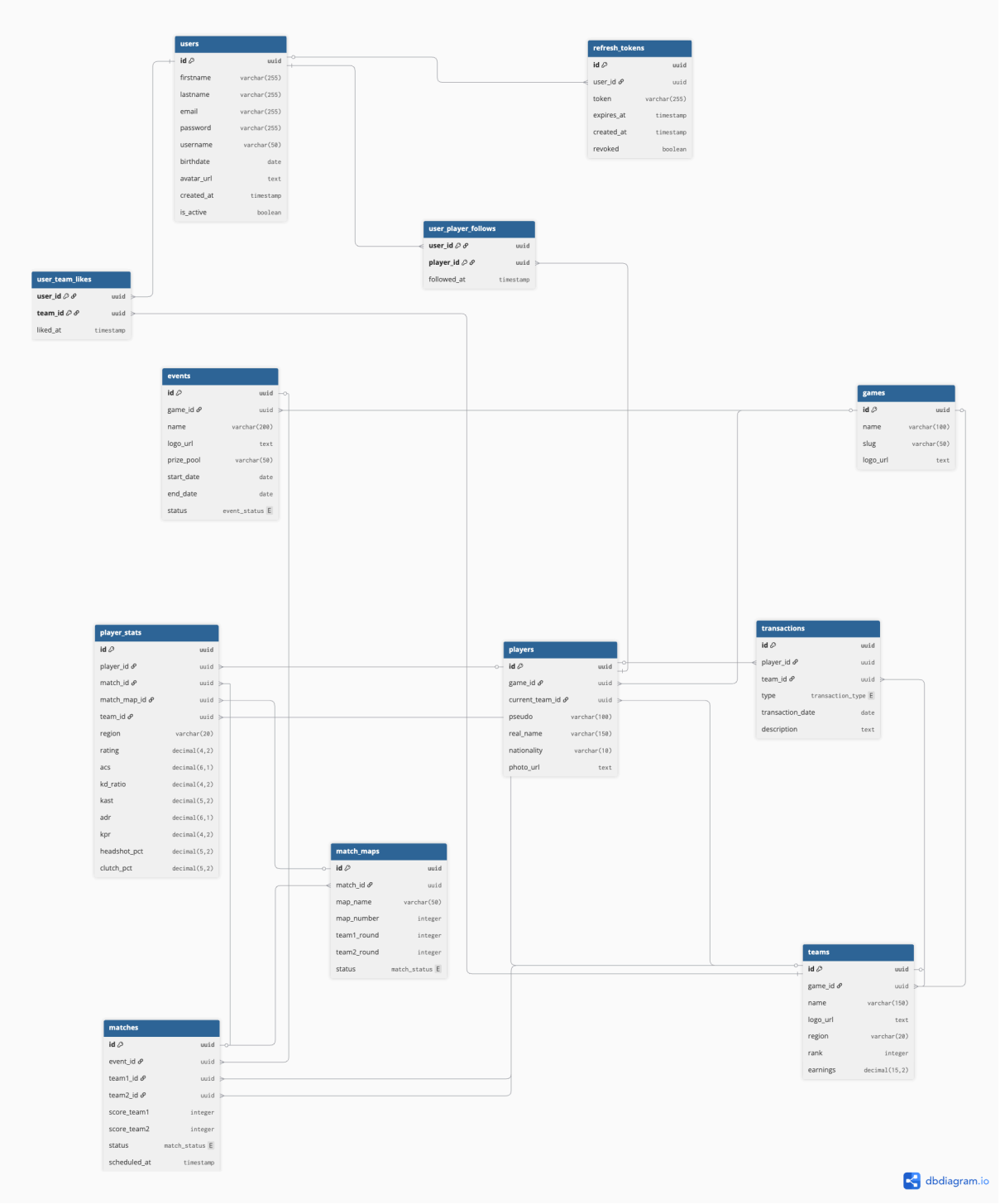
3. MCD



3. MLD

Quelques corrections ont été apportées lors de la conversion du MCD vers le MLD :

- Ajout de données liées au RGPD
- Création d’une table dédiée aux maps


users
id uuid
firstname varchar(255)
lastname varchar(255)
email varchar(255)
password varchar(255)
username varchar(50)
birthdate date
avatar_url text
created_at timestamp
is_active boolean
refresh_tokens
id uuid
user_id uuid
token varchar(255)
expires_at timestamp
created_at timestamp
revoked boolean
user_player_follows
user_id uuid
player_id uuid
followed_at timestamp
user_team_likes
user_id uuid
team_id uuid
liked_at timestamp
events
id uuid
game_id uuid
name varchar(200)
logo_url text
prize_pool varchar(50)
start_date date
end_date date
status event_status E
games
id uuid
name varchar(100)
slug varchar(50)
logo_url text
player_stats
id uuid
player_id uuid
match_id uuid
match_map_id uuid
team_id uuid
region varchar(20)
rating decimal(4,2)
acs decimal(6,1)
kd_ratio decimal(4,2)
kast decimal(5,2)
adr decimal(6,1)
kpr decimal(4,2)
headshot_pct decimal(5,2)
clutch_pct decimal(5,2)
players
id uuid
game_id uuid
current_team_id uuid
pseudo varchar(100)
real_name varchar(150)
nationality varchar(10)
photo_url text
transactions
id uuid
player_id uuid
team_id uuid
type transaction_type E
transaction_date date
description text
match_maps
id uuid
match_id uuid
map_name varchar(50)
map_number integer
team1_round integer
team2_round integer
status match_status E
teams
id uuid
game_id uuid
name varchar(150)
logo_url text
region varchar(20)
rank integer
earnings decimal(15,2)
matches
id uuid
event_id uuid
team1_id uuid
team2_id uuid
score_team1 integer
score_team2 integer
status match_status E
scheduled_at timestamp
dbdiagram.io